

Step 3 — Report Internal Progress Without Breaking the One-Tool Illusion — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real formatting code), **Advanced** (what to actually put in the trace, and what to leave out). Read Basic first even if the formatting feels obvious — the real decision in this step isn't the code, it's what belongs in the result.

- [Hint 1 — Deciding what the caller actually sees](#)
- [Hint 2 — Building the trace string](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Deciding What the Caller Actually Sees

Basic Version

`deep_research` still returns one plain string — that part never changes. What changes is what's *inside* that string. Right now it's just the final summary. After this step, it's the final summary plus a short list of what happened each round before it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You already have everything you need — `PipelineResult.rounds` is a list of `RevisionRound`, each with `round_number`, `summary`, `approved`, and `issues`. This step is mostly about writing one function that turns that list into readable text, and calling it from `deep_research` instead of returning `result.final_summary` directly.

```
def format_result(result):  
    # build a list of lines, then join them with "\n" at the end  
    ...
```

Building a list of strings and joining them once at the end (`"\n".join(lines)`) is cleaner than repeatedly concatenating with `+=` — it's the same pattern you'd use printing a multi-line report anywhere else.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The real question this step is testing is the one the README's "decision to make" box answers directly: should this be MCP's live progress-notification mechanism, or a trace baked into the result? Think about *why* the trace-in-result choice is the right one, not just *that* it's the choice — write your own one-paragraph answer before reading the README's reasoning again. Two angles worth thinking through yourself: what does a caller using the plain Inspector (no custom progress-listening code) actually see with each approach? And what's still true about the result five minutes after the call finishes, with each approach?

Also think about what a round's *summary text* looks like in a rejected round — should the trace include the full rejected draft, or just the fact that it was rejected and why? Including the full text of every rejected draft makes the trace long and noisy for a caller who mostly wants to know "did this work, and how much did it struggle" — leaving it out (just the round number, approved/rejected, and the issues) keeps the trace short while still being honest about what happened. This is the same "what's worth surfacing vs. what's just noise" judgment Doc11's shared-state section teaches, just applied to a result string instead of a state object.

Difference between Basic, Intermediate, and Advanced: Basic says what changes about the return value. Intermediate gives the real approach to building the formatted string. Advanced is the actual design decision underneath the code — which mechanism to use, and how much detail belongs in the trace — which matters far more here than the exact wording of your `format_result()` function.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — Building the Trace String

Basic Version

```
start with: "Deep research on: " + the topic

add a blank line, then "Revision trace:"
for each round:
    add " Round <number>: approved" or " Round <number>: rejected"
    for each issue in that round:
        add " - " + the issue

add a blank line, then a status line saying whether it was approved,
and after how many rounds (or that it hit the cap)

add a blank line, then "Summary:" and the final summary text

join all of that with newlines and return it
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
def format_result(result: PipelineResult) -> str:
    lines = [f"Deep research on: {result.topic}", "", "Revision trace:"]
    for round_info in result.rounds:
        status = "approved" if round_info.approved else "rejected"
        lines.append(f"  Round {round_info.round_number}: {status}")
        for issue in round_info.issues:
            lines.append(f"    - {issue}")

    lines.append("")
    if result.approved:
        lines.append(f"Status: approved after {result.revision_count} round(s).")
    else:
        lines.append(f"Status: NOT approved -- hit the {result.revision_count}-ro

    lines.append("")
    lines.append("Summary:")
    lines.append(result.final_summary)
    return "\n".join(lines)
```

Call this from `deep_research` instead of returning `result.final_summary` on its own: `return format_result(result)` .

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Test this against both the easy topic and the hard one from the README's Real Example. On the easy topic, the trace should be short — 1-2 rounds, one clear rejection reason, then approved. On the hard topic (the "almost nothing checkable" one), read the trace carefully: does it clearly tell you the pipeline hit the cap without approval, or does it read almost the same as a successful run if you're skimming? If a caller skimming the result can't immediately tell "this worked cleanly" from "this struggled and gave up," the status line needs to be more visually distinct — some real MCP servers put a marker like `[UNCONFIRMED]` or `⚠` right at the start of a not-approved result, specifically so a caller glancing at just the first line already knows something's off, without reading the whole trace.

Difference between Basic, Intermediate, and Advanced: Basic is the plain plan for the string. Intermediate is the real, working function. Advanced is testing your own output like a skeptical caller would — actually reading what comes back on the hard topic, and deciding whether "NOT approved" is loud enough to notice at a glance, not just technically present somewhere in the text.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)